



Instituto Federal de Educação, Ciência e Tecnologia da Paraíba
Campus Campina Grande
Coordenação do Curso Superior de Tecnologia em Telemática

IMPLEMENTAÇÃO E VALIDAÇÃO DE UMA MÁQUINA VIRTUAL DE LÓGICA LADDER E AGNÓSTICA DE HARDWARE BASEADA EM MODEL-BASED DESIGN

GABRIEL DOMINGOS DE MEDEIROS

Orientador: FAGNER DE ARAÚJO PEREIRA

Campina Grande, fevereiro de 2013

ⓂNome do aluno



Instituto Federal de Educação, Ciência e Tecnologia da Paraíba
Campus Campina Grande
Coordenação do Cursos Superior de Tecnologia em Telemática

IMPLEMENTAÇÃO E VALIDAÇÃO DE UMA MÁQUINA VIRTUAL DE LÓGICA LADDER E AGNÓSTICA DE HARDWARE BASEADA EM MODEL-BASED DESIGN

GABRIEL DOMINGOS DE MEDEIROS

Monografia apresentada à Coordenação do
Curso de Telemática do IFPB - Campus
Campina Grande, como requisito parcial
para conclusão do curso de Tecnologia em
Telemática.

Orientador: Fagner de Araújo Pereira

Campina Grande, fevereiro de 2013

IMPLEMENTAÇÃO E VALIDAÇÃO DE UMA MÁQUINA VIRTUAL DE LÓGICA LADDER E AGNÓSTICA DE HARDWARE BASEADA EM MODEL-BASED DESIGN

GABRIEL DOMINGOS DE MEDEIROS

FAGNER DE ARAÚJO

PEREIRA
Orientador

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Membro da Banca

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Membro da Banca

Campina Grande, Paraíba, Brasil
Fevereiro/2013

Dedicatória (OPCIONAL)

Alguma frase/pensamento (OPCIONAL)

Autor

Agradecimientos

XXXXXXXXXXXXXXXXXXXXXXX

Resumo

XX.

Palavras-chave: XXXXXXXXXXXXXXXXXXXXXXX.

Abstract

XX.

Keywords: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX.

Sumário

Lista de Abreviaturas	xii
Lista de Figuras	xiii
Lista de Tabelas	xiv
1 Introdução	1
1.1 Contextualização	1
1.2 Descrição do Problema	2
1.3 Objetivos	3
1.3.1 Objetivo Geral	3
1.3.2 Objetivos Específicos	3
1.4 Justificativa	4
1.5 Metodologia	4
1.6 Organização do Documento	5
2 Fundamentação Teórica	6
2.1 A Norma IEC 61131-3	6
2.2 Trabalhos Relacionados	6
2.2.1 Implementação da IEC 61131-3 sobre Sistemas POSIX de Tempo Real	7
2.2.2 Controladores Lógicos de Código Aberto	7
2.2.3 Geração e Validação Automática de Código via Model-Based Design .	8
2.2.4 Juliet & Romeo: Linguagem e VM para Automação Moderna	8
2.3 Estado da Prática	9
2.3.1 OpenPLC	9
2.3.2 CODESYS Runtime	9
2.3.3 Simulink PLC Coder	10
2.3.4 MicroPython	10
2.4 Conceitos Fundamentais de Arquitetura	11
3 Considerações Finais e Sugestões para Trabalhos Futuros	13
3.1 Sugestões para Trabalhos Futuros	13
A Base de Dados	14
Referências Bibliográficas	15

Lista de Abreviaturas

ARM	<i>Advanced RISC Machine</i>
CLP	Controlador Lógico Programável
FBD	<i>Function Block Diagram</i>
GPIO	<i>General Purpose Input/Output</i>
HAL	<i>Hardware Abstraction Layer</i>
IEC	<i>International Electrotechnical Commission</i>
IL	<i>Instruction List</i>
IoT	<i>Internet of Things</i>
ISA	<i>Instruction Set Architecture</i>
LD	<i>Ladder Diagram</i>
POSIX	<i>Portable Operating System Interface</i>
RAM	<i>Random Access Memory</i>
SFC	<i>Sequential Function Chart</i>
ST	<i>Structured Text</i>
VM	<i>Virtual Machine</i> (Máquina Virtual)
vPLC	<i>Virtual Programmable Logic Controller</i>

Lista de Figuras

Lista de Tabelas

1 INTRODUÇÃO

1.1 Contextualização

Este trabalho insere-se na interseção entre a Engenharia de *Software* Embarcado e a Automação Industrial, delimitando-se ao estudo e ao desenvolvimento de arquiteturas de *software* portáteis para o controle de processos. O tema central consiste na implementação de uma Máquina Virtual (*VM*) embarcada, projetada para interpretar instruções de lógica *Ladder* — baseada na norma IEC 61131-3 [International Electrotechnical Commission 2025] — de maneira independente da plataforma de *hardware* subjacente. Para viabilizar essa interpretação, o projeto engloba a definição de uma Arquitetura de Conjunto de Instruções (*ISA* — *Instruction Set Architecture*) customizada e de tamanho fixo (32 *bits*), projetada especificamente para otimizar o processo de busca e decodificação (*fetch-decode*) e garantir o determinismo do tempo de varredura.

A pesquisa foca especificamente na migração e na adaptação dessa arquitetura para microcontroladores da família *STM32* (núcleo *ARM Cortex-M7*), explorando técnicas de abstração de *hardware* (*HAL*) e metodologias de *Model-Based Design*. A relevância técnica do tema reside na investigação de métodos para desacoplar a lógica de controle industrial do *hardware* proprietário, promovendo a interoperabilidade e a redução de custos em sistemas de automação. Propõe-se a segregação estrutural entre o motor de execução (*firmware* estático) e a lógica de aplicação (*bytecode* tratado como dado), estabelecendo um ambiente de execução isolado (*sandbox*) que previne falhas críticas de processamento e viabiliza a gestão segura do ciclo de vida da aplicação industrial.

1.2 Descrição do Problema

O cenário atual de desenvolvimento de controladores lógicos industriais caracteriza-se por uma forte fragmentação tecnológica e pela predominância de soluções proprietárias, o que resulta no fenômeno de aprisionamento tecnológico. A rigidez das arquiteturas convencionais de Controladores Lógicos Programáveis (CLPs) impõe barreiras significativas à inovação, dificultando a portabilidade de algoritmos de controle para plataformas de *hardware* mais modernas e acessíveis sem a necessidade de reengenharia completa do *firmware*. Além disso, a validação de lógicas de controle complexas diretamente no *hardware* final apresenta riscos operacionais e custos elevados, carecendo de metodologias que integrem eficientemente a simulação computacional com a execução física.

No âmbito das soluções de código aberto atualmente disponíveis, como o ecossistema *OpenPLC* [Alves 2018], a abordagem predominante baseia-se na compilação de diagramas *Ladder* para código C nativo, seguida de compilação estática. Embora eficiente em termos de tempo de execução, essa metodologia funde a lógica de controle aos *drivers* de *hardware* em um binário monolítico. Consequentemente, a adaptação para novas plataformas ou pinagens exige modificações diretas no código-fonte dos *drivers* em baixo nível, limitando drasticamente a portabilidade. Adicionalmente, a execução do código traduzido diretamente no processador elimina barreiras de proteção inerentes a sistemas operacionais: um erro na modelagem da lógica do usuário pode resultar em falhas críticas do sistema, comprometendo a estabilidade do equipamento industrial.

Por outro lado, abordagens alternativas baseadas em interpretadores de linguagens de *script* genéricas, como o *MicroPython* [MicroPython 2025], falham em atender aos requisitos de tempo real estrito da automação industrial, devido à gestão não determinística de memória (*Garbage Collection*), que introduz variações inaceitáveis no tempo de varredura. O problema técnico a ser abordado, portanto, é a ausência de uma camada de virtualização eficiente e aberta que permita a execução segura e determinística de lógicas de controle padronizadas em microcontroladores de propósito geral, garantindo a fidelidade comportamental entre o modelo projetado em ambiente de simulação e o sistema embarcado final.

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver e validar uma máquina virtual embarcada capaz de interpretar *bytecode* de lógica *Ladder* com fidelidade semântica à norma IEC 61131-3, executando de forma determinística e agnóstica de *hardware* em um microcontrolador da família *STM32*.

1.3.2 Objetivos Específicos

- Especificar e documentar formalmente uma Arquitetura de Conjunto de Instruções (*ISA*) de 32 *bits*, compatível com as primitivas operacionais do subconjunto *Ladder Diagram* da IEC 61131-3;
- Modelar lógicas de controle de referência em ambiente *Simulink/Stateflow*, segundo a metodologia de *Model-Based Design*;
- Implementar, em linguagem C, o motor de execução (*VM*) responsável pelo ciclo de busca, decodificação e execução do *bytecode*;
- Desenvolver uma Camada de Abstração de *Hardware* (*HAL*) para o microcontrolador *STM32F7* (núcleo *ARM Cortex-M7*);
- Validar cruzadamente o comportamento da *VM* embarcada frente ao modelo de referência simulado, comparando as tabelas de imagem de saída para um mesmo conjunto de estímulos de entrada;
- Analisar quantitativamente o desempenho da solução, mensurando o tempo de varredura (*scan cycle*) e o consumo de memória *RAM* e *Flash*.

1.4 Justificativa

A relevância deste trabalho decorre da lacuna identificada entre as soluções de código aberto disponíveis e os requisitos de portabilidade e determinismo da automação industrial. Ao segregar o motor de execução da lógica de aplicação, a arquitetura proposta elimina a necessidade de recompilação do *firmware* a cada alteração da lógica *Ladder*, reduzindo o tempo de comissionamento e o risco de falhas decorrentes de atualizações completas de memória. O caráter de ambiente isolado (*sandbox*) confere ainda uma camada de segurança inexistente nas soluções que traduzem a lógica do usuário diretamente para código nativo, contendo eventuais erros estruturais antes que comprometam a estabilidade do sistema embarcado.

Do ponto de vista social e econômico, a proposta de uma solução aberta e portátil para múltiplas plataformas de *hardware* contribui para reduzir o aprisionamento tecnológico a fabricantes específicos, democratizando o acesso a tecnologias de automação hoje concentradas em soluções proprietárias de alto custo.

1.5 Metodologia

A metodologia adotada caracteriza-se como pesquisa aplicada de natureza experimental, estruturada em etapas sequenciais com entregas incrementais, seguindo uma abordagem *bottom-up* que parte da especificação formal da arquitetura até a validação funcional integrada ao *hardware*. Trata-se de pesquisa aplicada por visar à resolução de um problema técnico concreto — a ausência de uma camada de virtualização aberta e determinística para lógicas *Ladder* em microcontroladores de propósito geral — e de abordagem predominantemente quantitativa, uma vez que a validação da solução é aferida por métricas numéricas de desempenho, como o tempo de varredura e o consumo de memória. Os procedimentos técnicos configuram-se como pesquisa experimental, dado que envolvem a construção de um artefato de *software* embarcado e a manipulação controlada de variáveis durante os testes comparativos entre a simulação em *Simulink* e a execução física na *VM* embarcada.

O detalhamento de cada etapa metodológica — especificação da *ISA*, modelagem de referência, implementação do motor de execução, desenvolvimento da *HAL*, integração com

hardware, validação cruzada e análise de desempenho — será apresentado nos capítulos de desenvolvimento deste trabalho.

1.6 Organização do Documento

O restante deste trabalho está organizado da seguinte forma: o Capítulo 2 apresenta a fundamentação teórica, contemplando a norma IEC 61131-3, os trabalhos relacionados e o estado da prática em soluções para execução de lógicas de controle industrial, além dos conceitos de arquitetura que sustentam a solução proposta. Os capítulos subsequentes apresentarão a concepção e a modelagem detalhada da solução, os resultados obtidos na validação cruzada e na análise de desempenho, seguidos das considerações finais e das sugestões para trabalhos futuros, no Capítulo 3.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 A Norma IEC 61131-3

A norma IEC 61131-3, cuja quarta edição foi publicada em maio de 2025 pela Comissão Eletrotécnica Internacional (IEC), constitui a norma internacional que define a arquitetura de *software* e as linguagens de programação para controladores programáveis [International Electrotechnical Commission 2025]. A norma especifica a sintaxe e a semântica de um conjunto unificado de linguagens, incluindo *Structured Text (ST)*, *Ladder Diagram (LD)*, *Function Block Diagram (FBD)* e *Sequential Function Chart (SFC)*. A edição mais recente removeu a linguagem *Instruction List (IL)*, anteriormente marcada como obsoleta, e incorporou suporte a *strings UTF-8*. A importância da norma para o presente trabalho é fundamental, pois a *ISA* customizada da máquina virtual proposta é projetada para representar fielmente as primitivas operacionais definidas no subconjunto *LD* da IEC 61131-3, garantindo compatibilidade semântica com o padrão industrial.

2.2 Trabalhos Relacionados

No domínio de execução portátil de lógicas de controle industrial, destacam-se três eixos temáticos principais na literatura: a implementação do modelo de *software* da norma IEC 61131-3 em sistemas operacionais de tempo real, o desenvolvimento de controladores lógicos de código aberto e as metodologias de *Model-Based Design* aplicadas à geração automática de código para CLPs.

2.2.1 Implementação da IEC 61131-3 sobre Sistemas POSIX de Tempo Real

Plaza e Medrano [Plaza e Medrano 2006] apresentaram uma proposta de implementação do modelo de *software* definido pela norma IEC 61131-3 utilizando sistemas operacionais compatíveis com *POSIX* (*Pthreads* e temporizadores de alta resolução), especificamente o *RTLinux*. Os autores discutem os aspectos críticos da norma que necessitam de esclarecimento antes de sua implementação prática, como concorrência, escopo de variáveis e operação cíclica, demonstrando que o modelo IEC 61131-3 pode ser mapeado para primitivas *POSIX*, gerando código C a partir de descrições textuais em *Instruction List (IL)*. A contribuição central do trabalho reside na comprovação de que a camada de tradução de IEC para C no padrão *POSIX* é viável e que o desempenho resultante é comparável ao de produtos comerciais. Por outro lado, a abordagem permanece vinculada a plataformas com sistema operacional *Linux* em tempo real, não abordando a execução em microcontroladores *bare-metal* de recursos limitados, cenário de aplicação do presente trabalho.

2.2.2 Controladores Lógicos de Código Aberto

Alves [Alves 2018] apresentou o desenvolvimento do *OpenPLC*, o primeiro CLP de código aberto totalmente compatível com a norma IEC 61131-3. O trabalho detalha a arquitetura interna do sistema, composta pelo compilador *MatIEC* — responsável por traduzir código IEC 61131-3 para C++ —, servidor *web*, camadas de rede, tabelas de imagem de E/S e uma camada de *hardware*. Essa obra contribui significativamente para a identificação de limitações arquiteturais do *OpenPLC*: a compilação estática via *MatIEC* funde a lógica do usuário ao binário do sistema, eliminando a possibilidade de atualização dinâmica da lógica de controle e introduzindo riscos de falhas catastróficas caso a lógica do usuário contenha erros estruturais. Essas limitações motivam diretamente a arquitetura de segregação entre motor de execução e *bytecode* proposta neste trabalho.

2.2.3 Geração e Validação Automática de Código via Model-Based Design

Duggan et al. [Duggan *et al.* 2022] apresentaram uma metodologia para projeto e validação de equipamentos em ambientes de manufatura regulados, utilizando o *Simulink* como plataforma de *Model-Based Design*. A obra propõe um fluxo de trabalho no qual a lógica de controle é modelada graficamente, validada por simulação digital contra um modelo da planta — atuando como um gêmeo digital — e, em seguida, convertida automaticamente para blocos funcionais compatíveis com a IEC 61131-3 via *Simulink PLC Coder* [MathWorks 2025]. Os autores comprovam que a documentação de validação e o código *PLC* podem ser gerados automaticamente a partir do modelo do sistema, reduzindo significativamente o tempo de comissionamento. A metodologia de *Model-Based Design* apresentada por Duggan et al. [Duggan *et al.* 2022] é diretamente aplicável ao presente projeto como estratégia de verificação: a lógica de controle modelada em *Simulink* serve como referência para comparação com a execução na *VM* embarcada.

2.2.4 Juliet & Romeo: Linguagem e VM para Automação Moderna

De forma análoga à solução aqui proposta, a plataforma *Juliet & Romeo*, desenvolvida pela *Cognibotics* [Cognibotics 2025], representa uma abordagem emergente que combina uma linguagem de programação expressiva (*Juliet*) com uma máquina virtual de tempo real (*Romeo*). O sistema foi projetado para automação moderna e programação de movimentos, oferecendo garantias de tempo real e comportamento determinístico. Diferentemente da abordagem do presente projeto, que se concentra na interpretação de *bytecodes Ladder* em microcontroladores de recursos limitados, o *Juliet & Romeo* opera em plataformas de maior capacidade computacional e visa à coexistência com ambientes IEC 61131-3 existentes, como o *CODESYS* [CODESYS Group 2025], buscando atuar como extensão para funcionalidades avançadas de robótica e inteligência artificial. Não obstante, o conceito de separação entre linguagem de alto nível e máquina virtual de execução determinística é análogo ao proposto neste trabalho, ainda que a plataforma da *Cognibotics* divirja em abordagem, foco e cenário de aplicação.

2.3 Estado da Prática

O estado da prática em soluções reais para execução de lógicas de controle IEC 61131-3 compreende tanto produtos comerciais consolidados quanto projetos de código aberto. Esta seção descreve as principais soluções disponíveis no mercado e na comunidade, apontando suas características, pontos positivos e limitações.

2.3.1 OpenPLC

O *OpenPLC* [OpenPLC Project 2025] é a solução de código aberto mais madura para implementação de CLPs em *hardware* de propósito geral. Compatível com as cinco linguagens da IEC 61131-3, suporta execução em plataformas como *Raspberry Pi*, *Arduino* e PCs industriais com *Linux*. Sua arquitetura baseia-se no compilador *MatIEC*, que traduz o código IEC 61131-3 para C++ executado nativamente na plataforma embarcada selecionada, contando ainda com servidor *web* para *upload* e gerenciamento de programas, suporte ao protocolo *Modbus* e uma camada de *hardware* configurável. As limitações do *OpenPLC* residem na compilação estática, que gera um binário único no qual a lógica do usuário é fundida ao *firmware*, exigindo a regravação completa da lógica *Ladder* a cada alteração e impossibilitando atualizações dinâmicas; além disso, a portabilidade para novos *hardwares* exige modificações na camada de *drivers* em baixo nível.

2.3.2 CODESYS Runtime

O *CODESYS Runtime* [CODESYS Group 2025], desenvolvido pela empresa alemã *3S-Smart Software Solutions GmbH*, é o principal sistema de desenvolvimento IEC 61131-3 independente de fabricante, presente em mais de mil tipos diferentes de dispositivos de mais de quinhentos fabricantes. O sistema oferece variantes para PCs industriais com *Windows*,

dispositivos *Linux* com processadores *ARM* e controladores virtuais (*vPLCs*) executados em contêineres, gerando código nativo — e não interpretado — para uma ampla gama de processadores, o que garante desempenho superior. Por outro lado, o *CODESYS* é um produto comercial de licenciamento pago, representando uma barreira de custo significativa para projetos acadêmicos e aplicações de pequeno porte; seu código-fonte é proprietário e fechado, impedindo a inspeção e a modificação por pesquisadores, e a adaptação para novos *hardwares* requer a aquisição de um *Runtime Toolkit* proprietário.

2.3.3 Simulink PLC Coder

O *Simulink PLC Coder* [MathWorks 2025], extensão da *MathWorks* para o *MATLAB*, gera automaticamente código IEC 61131-3 a partir de modelos *Simulink*, gráficos *Stateflow* e funções *MATLAB*. O código gerado é exportado em formatos compatíveis com os principais IDEs industriais, incluindo *CODESYS*, *Studio 5000*, *TIA Portal* e *Sysmac Studio*, produzindo também bancos de teste para verificação da correspondência entre a simulação e a execução no CLP físico, além de relatórios de geração de código com rastreabilidade bidirecional entre modelo e código. A principal limitação dessa solução é pressupor a existência de um CLP alvo convencional ou *Soft PLC* para a execução do código gerado, não oferecendo uma camada de virtualização própria para microcontroladores de propósito geral; adicionalmente, as licenças *MATLAB/Simulink* e do *PLC Coder* representam um investimento financeiro significativo.

2.3.4 MicroPython

Uma abordagem alternativa observada na prática consiste na utilização de interpretadores de linguagens de *script* genéricas, como o *MicroPython* [MicroPython 2025], em microcontroladores para implementação de lógicas de controle simples. O *MicroPython* executa *bytecodes Python* em uma máquina virtual compatível com plataformas como *ESP32*, *STM32* e *Raspberry Pi Pico*. Embora ofereça portabilidade e facilidade de programação, a gestão de memória por *Garbage Collection* introduz pausas não determinísticas que violam

os requisitos de temporização estrita do ciclo de varredura; além disso, a linguagem *Python* não foi projetada para expressar nativamente primitivas de lógica *Ladder*. Dessa forma, o *MicroPython* atende satisfatoriamente aplicações de *IoT* e prototipagem, mas não oferece as garantias de determinismo temporal necessárias para controle industrial.

2.4 Conceitos Fundamentais de Arquitetura

Esta seção introduz os conceitos teóricos que fundamentam as decisões de projeto detalhadas nos capítulos de desenvolvimento deste trabalho.

Uma Arquitetura de Conjunto de Instruções (*ISA*) de largura fixa define instruções que ocupam sempre o mesmo número de *bits* em memória. Essa escolha simplifica o estágio de busca e decodificação (*fetch-decode*) de um interpretador: como o avanço do ponteiro de instrução é constante, o tempo de decodificação torna-se previsível, característica desejável para sistemas com requisitos de determinismo temporal, como o ciclo de varredura de um CLP. O ciclo *fetch-decode-execute* constitui o modelo clássico de operação de uma máquina virtual ou processador: a instrução é buscada na memória de programa, decodificada para identificar a operação e os operandos, e então executada, repetindo-se o processo para a instrução seguinte.

A Camada de Abstração de *Hardware* (*HAL*) é um padrão arquitetural que encapsula o acesso aos periféricos físicos de um dispositivo — como *GPIOs*, temporizadores e conversores analógico-digitais — por trás de uma interface estável e independente do *hardware* subjacente. A adoção de uma *HAL* permite que a lógica de aplicação e o motor de execução permaneçam inalterados quando o projeto é portado para uma nova plataforma, restringindo o esforço de adaptação à reimplementação da própria camada.

Um ambiente de execução isolado (*sandbox*) é um mecanismo que restringe o escopo de atuação de um programa em execução, validando em tempo real os limites de seus acessos — como endereços de memória e índices de entrada e saída — e rejeitando operações inválidas. Em sistemas embarcados críticos, esse mecanismo contém erros estruturais na lógica do usuário antes que eles comprometam a estabilidade do restante do sistema.

Por fim, o *Model-Based Design* é uma metodologia de desenvolvimento na qual um modelo executável do sistema — e, quando aplicável, da planta controlada — é utilizado como artefato central ao longo de todo o ciclo de vida do projeto, desde a especificação até a verificação. Nesse paradigma, a simulação do modelo serve como referência comportamental

para validar a implementação final, estratégia adotada neste trabalho para a validação cruzada entre a lógica de controle modelada em *Simulink/Stateflow* e sua execução na máquina virtual embarcada.

3 CONSIDERAÇÕES FINAIS E SUGESTÕES PARA TRABALHOS FUTUROS

3.1 Sugestões para Trabalhos Futuros

A BASE DE DADOS

Capítulo opcional usado para que o autor coloque um texto, documento ou informação referente ao assunto do trabalho, mas que não deve interromper a leitura.

Referências Bibliográficas

- [Alves 2018] ALVES, T. OpenPLC: An IEC 61131-3 compliant open source industrial controller for cyber security research. *Computers & Security*, v. 78, p. 364–379, 2018. 2, 7
- [CODESYS Group 2025] CODESYS Group. *CODESYS Runtime: the adaptable runtime environment*. 2025. Disponível em: <<https://www.codesys.com/products/runtime/>>. Acesso em: 2025. 8, 9
- [Cognibotics 2025] Cognibotics. *Juliet & Romeo® and IEC 61131: How They Work Together*. 2025. Disponível em: <<https://www.cognibotics.com/en/articles/juliet-romeo-and-iec-61131>>. Acesso em: 2025. 8
- [Duggan et al. 2022] DUGGAN, J. et al. A model-based approach to automated validation and generation of PLC code for manufacturing equipment in regulated environments. *Applied Sciences*, v. 12, n. 15, 2022. Art. 7506. 8
- [International Electrotechnical Commission 2025] International Electrotechnical Commission. *IEC 61131-3:2025 — Programmable controllers — Part 3: Programming languages*. 4. ed. Geneva, 2025. 1, 6
- [MathWorks 2025] MathWorks. *Simulink PLC Coder: Generate IEC 61131-3 Structured Text and Ladder Diagrams*. 2025. Disponível em: <<https://www.mathworks.com/products/simulink-plc-coder.html>>. Acesso em: 2025. 8, 10
- [MicroPython 2025] MicroPython. *MicroPython — Python for microcontrollers*. 2025. Disponível em: <<https://micropython.org/>>. Acesso em: 2025. 2, 10
- [OpenPLC Project 2025] OpenPLC Project. *OpenPLC Runtime v4*. 2025. Disponível em: <<https://github.com/Autonomy-Logic/openplc-runtime>>. Acesso em: 2025. 9
- [Plaza e Medrano 2006] PLAZA, I.; MEDRANO, C. Analysis and implementation of the IEC 61131-3 software model under POSIX real-time operating systems. *Microprocessors and Microsystems*, v. 30, n. 8, p. 497–508, 2006. 7